Sequential RISC-V Processor Design in Verilog

Introduction to Processor Architecture — Project Phase I

Group 5 Team 31

Member 1: Subal Manchanda (2024102033)
Member 2: Anish Toshniwal (2024102009)
Member 3: Parth Desai (2024102049)

February 2026

Contents

1 Abstract

This project involves the design and implementation of a Sequential 64-bit RISC-V processor using Verilog. The processor supports a subset of the RV64I instruction set including `add`, `sub`, `addi`, `and`, `or`, `ld`, `sd`, and `beq`. Each instruction executes in a single cycle. The processor is simulated using `iVerilog`, and the results are analyzed through `GTKWave`. This report presents the design details, implementation methodology, and verification of the processor modules.

2 Introduction and the Sequential datapath

The Sequential RISC-V processor executes one instruction at a time, ensuring that each instruction completes fully before the next begins. This design is simpler than the pipelined version and serves as a foundation for understanding instruction flow, datapath design, and control signal generation. However, it is slower and forces the use of a longer clock period and lower clock frequency. This is because all the stages of the processor have to be completed for an instruction before fetching the next one. Hence, the maximum frequency of the clock could be $f_{max} = \frac{1}{t_{fetch}+t_{decode}+t_{execute}+t_{memory}+t_{write}}$

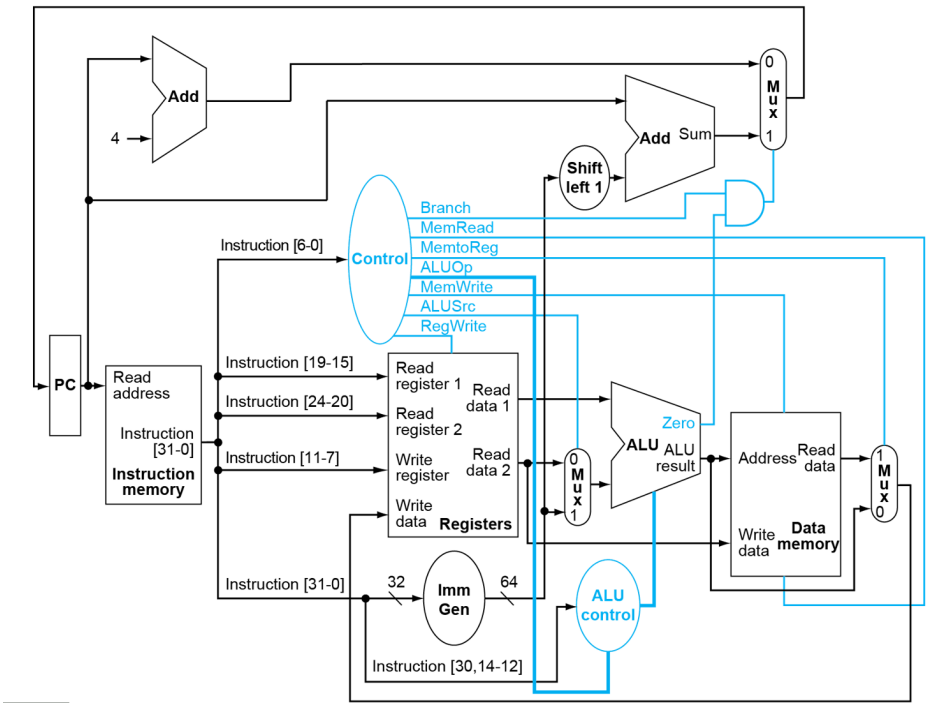

Figure 1: Sequential datapath of RISC-V

3 Design Specifications and Modules

Each component of the datapath was implemented as a separate Verilog module and tested independently. The design follows a modular and hierarchical structure to simplify debugging and integration.

3.1 Program Counter (PC)

Inputs: clk, reset, pc_in
Outputs: pc_out

The Program Counter has the address of the current instruction. It increments by 4 after each instruction, unless a branch occurs. On reset, the PC value is set to 0. Write signal is not needed as PC is updated with value of pc_in for every clock cycle.

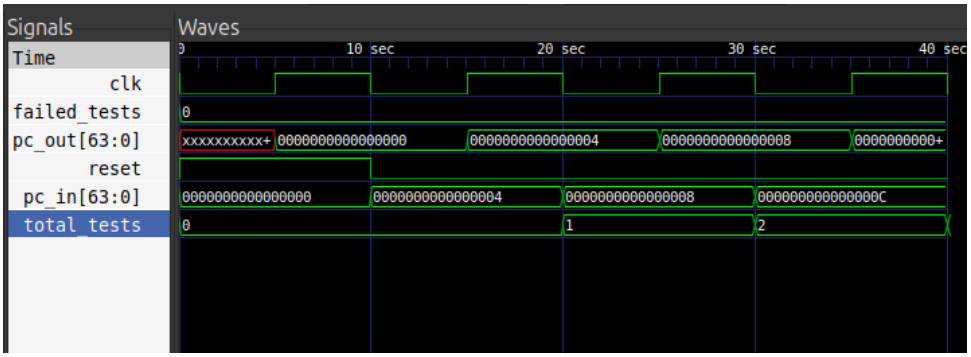

Figure 2: GTKWave output for program counter

3.2 Register File

Inputs: clk, reset, read_reg1, read_reg2, write_reg, write_data, reg_write_en
Outputs: read_data1, read_data2

The register file is made up of 32 registers, all initialized to zero at the start. The register x0 is hardwired to always stay at zero. From each 32-bit instruction, two 5-bit fields are extracted - these represent the source register addresses read_reg1 and read_reg2 (when both are used). The data from these registers are output as read_data1 and read_data2. Here, read_data1 directly feeds into the ALU's first input, while read_data2 passes through a 2×1 multiplexer, which chooses between the register value and the immediate value based on the control signal. Reading happens continuously, but writing to a register only occurs when reg_write_en is active. The write_data signal, which is 64 bits wide, carries the value that will be stored into the target register specified by write_reg.

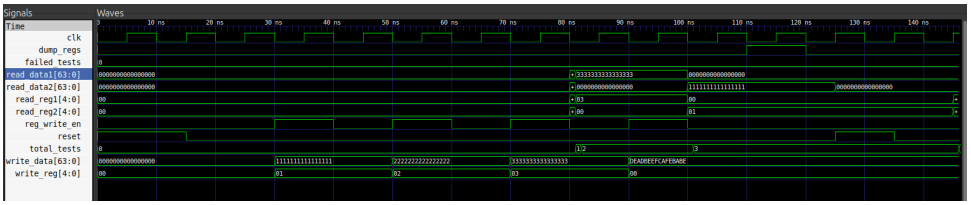

Figure 3: Register file GTKwave

3.3 Instruction Memory

Inputs: addr
Outputs: instr

The instruction memory reads 32-bit instructions from a text file named `instructions.txt`, stored in Big-Endian byte format. It provides instructions corresponding to the current PC value. It is treated as combinational as we do not write to instruction memory and hence no read control signal is needed.

Figure 4: Instruction Memory GTKwave

3.4 Control Unit

Input: opcode
Outputs: Branch, MemtoReg, MemRead, ALUOp[1:0], MemWrite, ALUSrc, RegWrite

The Control Unit generates control signals based on the instruction type. It decodes the opcode to produce control signals for the ALU, Register File, and Memory blocks.

- Branch is set if the instruction is a branch instruction
- MemtoReg is set if the instruction is a load instruction for sending data from data memory to registers in the write back stage. It is reset if data written to registers in write back stage has to be taken from the ALU.
- MemRead is set if the instruction is a load instruction for reading from memory
- MemWrite is set if the instruction is a store instruction for writing into data memory.
- RegWrite is set if the instruction requires a write back, like in the case of a load or R-type instruction.
- ALUSrc is set if the second input of the ALU is taken from the immediate block. Else it is taken from read_data2.
- ALUOp is set as 10 for R-type instructions 00 for I and S type instructions and 01 for branch instructions.

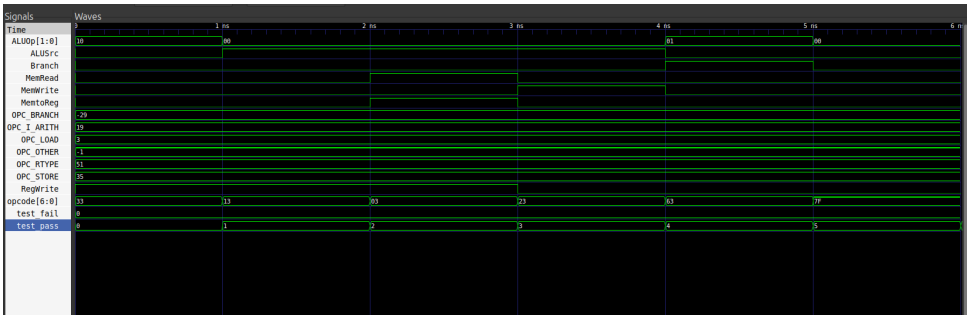

Figure 5: Control signal waveform

3.5 Immediate Generation

Input: instruction
Outputs: 64 bit sign extended immediate value

This module extracts and sign-extends the immediate value from the instruction depending on the format (I, S, or B type).

- **I-type:** Bits [31:20] : The immediate value is extracted from bits 31-20 and its sign is extended to 64 bits
- **S-type:** Bits [31:25] and [11:7] : The bits are extracted from 31-25 and 11-7 and concatenated, then sign extended to 64 bits
- **B-type:** mm[12] (bit 31), imm[10 : 5] (bits 30-25), imm[4 : 1] (bits 11-8), imm[11] (bit 7) : This is how the immediate value bits are formed from the instruction. Then, it is sign extended to 64 bits and one shift to the left is done to make it word aligned.

Figure 6: Immediate Generation GTKwave

3.6 ALU Control

Input: 2 bit ALUOp , instruction[30, 14-12]
Outputs: 4 bit ALUControl

Generates the 4-bit control signal for the ALU based on ALUOp and function bits. The logic distinguishes between arithmetic and branch instructions. The 4 bit ALUControl

specifies the exact ALU instruction which allows other formats to have same ALU operation when needed.

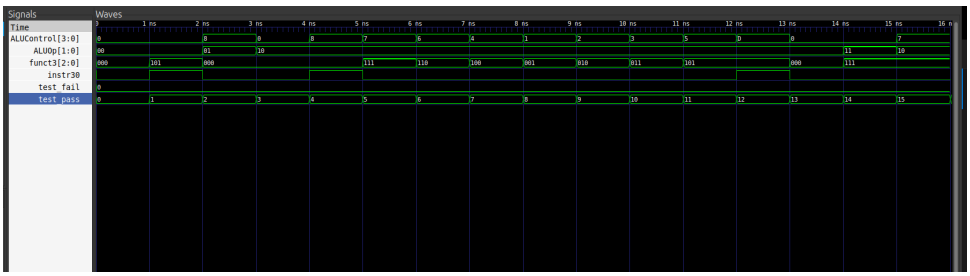

Figure 7: ALU Control GTKwave

3.7 Arithmetic Logic Unit (ALU)

Inputs: input1, input2, control
Outputs: result, zero flag

Performs arithmetic and logical operations like add, sub, and, or. Sets the zero flag used for branch decisions.

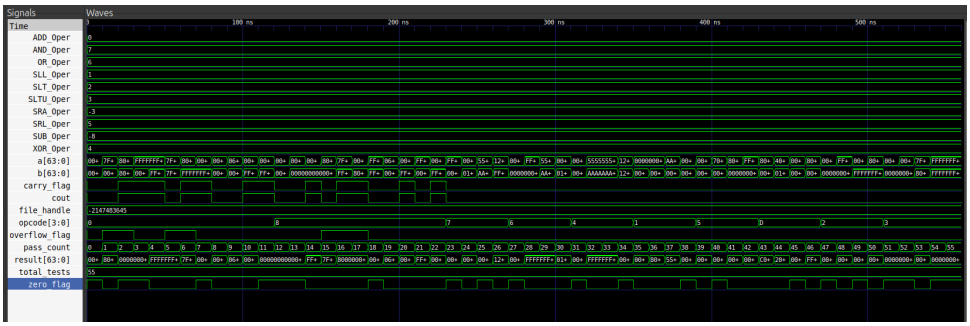

Figure 8: ALU waveform output

3.8 Data Memory

Inputs: clk, reset, address, write_data, MemRead, MemWrite
Outputs: read_data

Stores and retrieves 64-bit data in a 1024-byte memory. Uses a 10-bit address and supports one-cycle read/write latency. Initially all set to 0. If MemWrite is high, the value in Write_data is written to memory at specified address. If MemRead is high then data from the given address is read and stored in read_data.

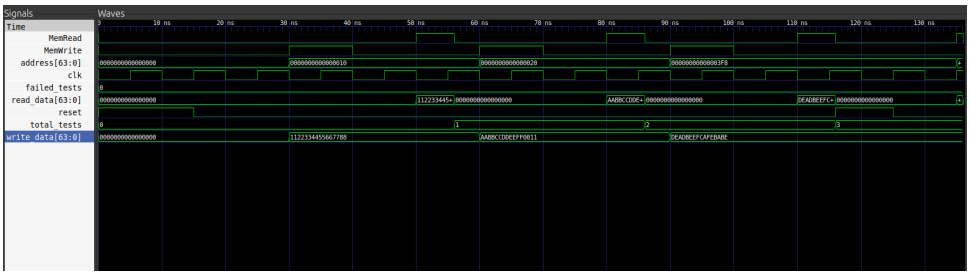

Figure 9: GTKWave output for Data Memory

3.9 Multiplexers and Adders

Multiple MUXes are used to select between ALU inputs and WriteBack data. Adders are used to compute PC + 4 and branch target addresses.

4 Integration: The Sequential Processor

All individual modules were integrated to form the complete Sequential Processor. The processor executes one instruction per clock cycle and produces the expected results for all supported instructions.

5 Testing and Verification

Testing was done using Verilog testbenches and assembly programs converted to machine code. Each test produces an output file `register_file.txt` showing final register states and clock cycles.

5.1 Test Programs

- **Sum of N Numbers:** Calculates the sum of the first N natural numbers.
  This is the human readable assembly:

```
addi x1, x0, 25
addi x2, x0, 0
addi x3, x0, 1
add  x10, x3, x0
add  x2, x2, x3
addi x3, x3, 1
sub  x4, x3, x1
beq  x4, x10, 8
beq  x0, x0, -16
```

  The results for this are given below, these are all the non-zero registers:

Table 1: Register Values after Execution

Register	Value (Decimal)	Description
x1	25	n
x2	325	sum
x3	26	n + 1
x4	1	
x10	1	

- **Fibonacci Sequence:** Generates the first few Fibonacci numbers.
  This is the human readable assembly:

```
        # first 10 fibonacci numbers

addi x1, x0, 10
addi x2, x0, 0
addi x3, x0, 0
addi x4, x0, 1

sd   x3, 0(x2)
addi x2, x2, 8
sd   x4, 0(x2)
addi x2, x2, 8

addi x5, x0, 2

beq  x5, x1, 32
add  x6, x3, x4
sd   x6, 0(x2)

addi x2, x2, 8
addi x5, x5, 1
add  x3, x0, x4
add  x4, x0, x6

beq  x0, x0, -28

addi x2, x0, 0
ld   x10, 0(x2)
ld   x11, 8(x2)
ld   x12, 16(x2)
ld   x13, 24(x2)
ld   x14, 32(x2)
ld   x15, 40(x2)
ld   x16, 48(x2)
ld   x17, 56(x2)
ld   x18, 64(x2)
ld   x19, 72(x2)
```

The results for this are given below, these are all the non-zero registers:

Table 2: Register Values after Fibonacci Sequence Execution

Register	**Value (Decimal)**	**Description**
x1	10	Loop limit / n
x3	21	Intermediate result
x4	34	Final Fibonacci value
x5	10	Counter variable
x6	34	Result mirror register
x11	1	Fibonacci term 1
x12	1	Fibonacci term 2
x13	2	Fibonacci term 3
x14	3	Fibonacci term 4
x15	5	Fibonacci term 5
x16	8	Fibonacci term 6
x17	13	Fibonacci term 7
x18	21	Fibonacci term 8
x19	34	Fibonacci term 9

6 Challenges and Observations

- Correct reconstruction of B-type immediate
- Accurate PC-relative branch offset calculation
- Maintaining consistent Big-endian memory format
- Preventing infinite loops due to incorrect branch offsets
- Proper byte addressing for 64-bit load/store operations